Cours du 11 Decembre 2025
#####################
TURTLE - virtualBox.ps1
###################

```
# Function to print a message and stop the script if an error occurs
function Stop-ScriptOnError {
    param (
        [string]$message
    )
    Write-Error $message
    exit 1
}

# Disable Hyper-V
try {
    Write-Output "Disabling Hyper-V (bcdedit)..."
    cmd.exe /c bcdedit /set hypervisorlaunchtype off
    Write-Output "Successfully disabled Hyper-V."
} catch {
    Stop-ScriptOnError "Failed to disable Hyper-V."
}

# Disable Hyper-V, part two
try {
    Write-Output "Disabling Hyper-V (DISM)..."
    DISM /Online /Disable-Feature:Microsoft-Hyper-V
    Write-Output "Successfully disabled Hyper-V (DISM)."
} catch {
    Stop-ScriptOnError "Failed to disable Hyper-V (DISM)."
}

# Disable Device Guard and Credential Guard
try {
    Write-Output "Disabling Device Guard and Credential Guard..."
    $regPathSystemGuard =
"HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\SystemGuard"
    $regPathCredentialGuard =
"HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard\Scenarios\CredentialGua
rd"
    $regPathDeviceGuard =
"HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard"

    if (Test-Path $regPathSystemGuard) {
        Set-ItemProperty -Path $regPathSystemGuard -Name "Enabled" -Value 0
        Write-Output "Successfully disabled Device Guard."
    } else {
        Write-Warning "Device Guard registry path not found, ignoring."
    }

    if (Test-Path $regPathCredentialGuard) {
```

```
        Set-ItemProperty -Path $regPathCredentialGuard -Name "Enabled" -
Value 0
        Write-Output "Successfully disabled Credential Guard."
    } else {
        Write-Warning "Credential Guard registry path not found, ignoring."
    }

    if (Test-Path $regPathDeviceGuard) {
        Set-ItemProperty -Path $regPathDeviceGuard -Name
"EnableVirtualizationBasedSecurity" -Value 0
        Write-Output "Successfully disabled
EnableVirtualizationBasedSecurity."
    } else {
        Write-Warning "EnableVirtualizationBasedSecurity registry path not
found, ignoring."
    }
} catch {
    Stop-ScriptOnError "Failed to disable Device Guard or Credential
Guard."
}

# Disable Virtual Secure Mode
try {
    Write-Output "Disabling Virtual Secure Mode..."
    cmd.exe /c bcdedit /set vsmlaunchtype Off
    Write-Output "Successfully disabled Virtual Secure Mode."
} catch {
    Stop-ScriptOnError "Failed to disable Virtual Secure Mode."
}

Write-Output "All steps completed. Please verify that all changes were
applied successfully."
Write-Output "Complete manual steps if necessary, and reboot the system."
```

##

Liste des commandes POWERSHELL

<u>Le partage</u>

Get-smbshare => Liste des partages réseaux
New-smbshare => Creer un nouveau partage
Remove-smbshare => supprime le partage
Grant-smbshareaccess = Modifier les autorisations de partage
revoke-smbshareaccess = Supprimer les autorisations de partage
Get-smbshareaccess => liste les autorisations de partage
new-smbmapping => Mapper un lecteur réseau

Exemple

Partage un dossier
New-smbshare -path "e:\partage" -name "partagewin10" -ReadAccess "toto","titi" -FullAccess "tutu"

Controle total => fullaccess
Lecture => readaccess
Modifer => Modifyaccess

#Ajouter des permissions
Grant-smbshareaccess -name "partagewin10" -AccountName "gege" -AccessRight Full -force
Controle total => full
Lecture => read
Modifer => change

#Supprimer des permission
Revoke-smbshareaccess -name "partagewin10" -AccountName "gege" -force
Controle total => full
Lecture => read
Modifer => change

#Supprimer un partage reseau
Remove-smbshare -name "partagewin10"

#Mapper un lecteur
new-smbmapping -localpath 'p:' -Remotepath '\\ad01\partage'

TP
Faire un script qui demande à l'utilisateur le nom d'un dossier
Il crée le dossier sur la partition e:
Il partage le dossier et donne contrôle total à "tout le monde"
Affiche la liste des dossiers partagés de la machine (le nom de partage et le chemin du dossier)

```
$dossier = read-host "Veuillez saisir le nom du dossier"
new-item -Path "e:\$dossier\" -Name "$dossier" -ItemType Directory
New-smbshare -path "e:\$dossier" -name "$dossier" -FullAccess "tout le
monde"
Get-smbshare  | Format-List -Property $_.path
read-host
```

Faire un script Powershell qui affiche les informations suivantes
=== Informations Réseau Wi-Fi ===
Interface : Wi-Fi
IPv4 locale : 172.20.0.110
IPv6 locale : fe80::6d9f:5f18:52db:2c48%13
IP publique : 194.206.218.105

****

rezo.ps1
****

```
# Récupération de l'interface Wi-Fi (nom contenant "Wi*fi")
$wifi = Get-NetIPAddress |
    Where-Object { $_.InterfaceAlias -like "Wi*fi" }

# Extraction IPv4 / IPv6
$ipv4 = $wifi | Where-Object { $_.AddressFamily -eq "IPv4" } | Select-
Object -ExpandProperty IPAddress
$ipv6 = $wifi | Where-Object { $_.AddressFamily -eq "IPv6" } | Select-
Object -ExpandProperty IPAddress

$ipPublic = Invoke-RestMethod -Uri "https://api.ipify.org?format=text"

# Affichage
Write-Host "=== Informations Réseau Wi-Fi ===" -ForegroundColor Cyan
Write-Host "Interface    : $($wifi[0].InterfaceAlias)"
Write-Host "IPv4 locale  : $ipv4"
Write-Host "IPv6 locale  : $ipv6"
Write-Host "IP publique  : $ipPublic"

read-host
```

==========

****
menu.ps1
****

```
$continue = $true
while ($continue) {
    write-host ".............MON MENU..............."
    write-host "1 - Action 1"
    write-host "2 - Action 2"
    write-host "3 - Action 3"
    write-host "q - Quitter le menu"
    write-host "................................."
    $choix=read-host "Veuillez selection votre action"
    switch ($choix) {
        1{Write-host "Vous avez choisi le menu 1"}
        2{Write-host "Vous avez choisi le menu 2"}
        3{Write-host "Vous avez choisi le menu 3"}
        'q'{$continue = $false}
        default {Write-host "Je n'ai pas compris votre choix"}
    }
}
```

****
menu-rezo.ps1

****

```
# Récupération de l'interface Wi-Fi (nom contenant "Wi*fi")
$wifi = Get-NetIPAddress |
    Where-Object { $_.InterfaceAlias -like "Wi*fi" }

# Extraction IPv4 / IPv6
$ipv4 = $wifi | Where-Object { $_.AddressFamily -eq "IPv4" } | Select-
Object -ExpandProperty IPAddress
$ipv6 = $wifi | Where-Object { $_.AddressFamily -eq "IPv6" } | Select-
Object -ExpandProperty IPAddress
$ipPublic = Invoke-RestMethod -Uri "https://api.ipify.org?format=text"

$continue = $true
while ($continue) {
    write-host "............MON MENU..............."
    write-host "1 - Afficher iPV4 de carte Wifi"
    write-host "2 - Afficher iPV6 de carte Wifi"
    write-host "3 - Afficher IP Publique de carte Wifi"
    write-host "q - Quitter le menu"
    write-host "................................."
    $choix=read-host "Veuillez selection votre action"
    switch ($choix) {
        1{Write-Host "IPv4 locale  : $ipv4"}
        2{Write-Host "IPv6 locale  : $ipv6"}
        3{Write-Host "IP publique  : $ipPublic"}
        'q'{$continue = $false}
        default {Write-host "Je n'ai pas compris votre choix"}
    }
}
```

****

FAIRE UN MENU QUI AFFICHE LES INFOS SUIVANTES

1- * VERSION SYSTEME EXPLOITATION (Avec la version release ex: 25h2)
2 - * DEPUIS COMBIEN DE TEMPS LE PC EST ALLUME
3 - * TAILLE DES DISQUES DUR ET L ESPACE UTILISE
4 - * IP V4 CARTE WIFI
5 - * IP PUBLIC
6 - * TAUX OCCUPATION DU PROCESSEUR
7 - * LA RAM ET LE TAUX OCCUPATION DE LA RAM
Q - * QUITTER LE MENU

```
# ============================================================
#   SCRIPT D’INFORMATION SYSTEME
# ============================================================
```

```
$continue = "ok"

while ($continue -eq "ok") {
    write-host ".............MON MENU..............."
 write-host "1-  * VERSION SYSTEME EXPLOITATION (Avec la version release
ex: 25h2 )"
 write-host "2 - * DEPUIS COMBIEN DE TEMPS LE PC EST ALLUME"
 write-host "3 - * TAILLE DES DISQUES DUR ET L ESPACE UTILISE"
 write-host "4 - * IP V4 CARTE WIFI"
 write-host "5 - * IP PUBLIC"
 write-host "6 - * TAUX OCCUPATION DU PROCESSEUR"
 write-host "7 - * LA RAM ET LE TAUX OCCUPATION DE LA RAM"
 write-host "Q - * QUITTER LE MENU"
    write-host "................................."
    $choix=read-host "Veuillez selection votre action"
    switch ($choix) {
        1{
            cls
            $os = Get-CimInstance Win32_OperatingSystem
            $win = Get-ItemProperty "HKLM:\SOFTWARE\Microsoft\Windows
NT\CurrentVersion"
            Write-Host "OS :" $os.Caption - $win.DisplayVersion -
ForegroundColor Red
      }
        2{
            cls
            $uptime = (Get-Date) - (Get-CimInstance
Win32_OperatingSystem).LastBootUpTime
            Write-Host ("Le PC est allumé depuis : {0} jours {1} heures {2}
minutes" -f `
            $uptime.Days, $uptime.Hours, $uptime.Minutes) -ForegroundColor
Red
      }
        3{
            cls
            $disques = Get-CimInstance Win32_LogicalDisk | Where-Object {
$_.DriveType -eq 3 }
            foreach ($d in $disques) {
                  $tailleTotale = [math]::Round($d.Size / 1GB, 2)
                  $libre = [math]::Round($d.FreeSpace / 1GB, 2)
                  $utilise = [math]::Round(($d.Size - $d.FreeSpace) / 1GB,
2)

                  Write-Host "Lecteur $($d.DeviceID) : Total=$tailleTotale
Go | Utilise=$utilise Go | Libre=$libre Go" -ForegroundColor Red
            }
      }
        4{
            cls
```

```
            $wifi = Get-NetIPAddress | Where-Object { $_.InterfaceAlias -
like "Wi*fi" }

            $ipv4 = $wifi | Where-Object { $_.AddressFamily -eq "IPv4" } |
Select-Object -ExpandProperty IPAddress
            Write-Host "IPv4 locale  : $ipv4" -ForegroundColor Red
      }
        5{
            cls
            $ipPublic = Invoke-RestMethod -Uri
"https://api.ipify.org?format=text"
            Write-Host "IP PUBLIC  : $ipPublic" -ForegroundColor Red
      }
        6{
            cls
            $cpu = Get-CimInstance Win32_Processor | Select-Object -
ExpandProperty LoadPercentage
            write-Host "Charge CPU : $cpu %" -ForegroundColor Red

      }
        7{
            cls
            # Récupération des informations mémoire
            $mem = Get-CimInstance Win32_OperatingSystem

            # Total et libre en Mo → converti en Go
            $totalRAM = [math]::Round($mem.TotalVisibleMemorySize / 1MB, 2)
            $freeRAM  = [math]::Round($mem.FreePhysicalMemory / 1MB, 2)
            $usedRAM  = [math]::Round($totalRAM - $freeRAM, 2)

            # Taux d'utilisation en pourcentage
            $ramPercent = [math]::Round(($usedRAM / $totalRAM) * 100, 1)

            # Affichage
            Write-Host " - Total     : $totalRAM Go" -ForegroundColor Red
            Write-Host " - Utilisée  : $usedRAM Go" -ForegroundColor Red
            Write-Host " - Libre     : $freeRAM Go" -ForegroundColor Red
            Write-Host " - Taux occ. : $ramPercent %" -ForegroundColor Red

      }
      'q'{$continue = "toto"}
        default {Write-host "Je n'ai pas compris votre choix"}
    }
}
```